Assignment- VIII

Group Project: Initial Proposal

Project title: 32-bit RISC-V Processor with a Five-Stage Pipeline

1. Team Details

Name: Shiv Sriram V
Registration number: 24BVD0168
Lab slot: L21+L22

Existing RISC-V v2 project: https://github.com/Triumphant-24k/riscv-r32-processor-v2

2. Project Proposal

Problem Definition

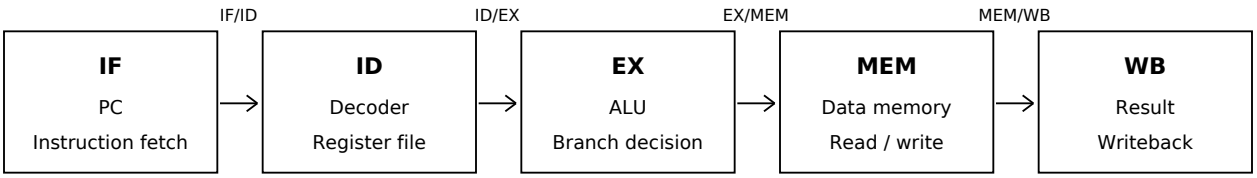
I have already worked on a 32-bit RISC-V processor. For this project, I plan to add pipelining to the processor and take the updated RTL through the RTL-to-GDSII flow. Pipelining allows different instructions to be in different stages at the same time. The main challenge is making sure the processor still gives the correct result when instructions depend on each other or when a branch changes the program flow.

Proposed Design

I will use five stages: instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM) and writeback (WB). Registers between stages will hold the instruction data and control signals. The processor will include a program counter, register file, decoder, immediate generator, ALU and instruction/data memory interfaces.

I plan to add forwarding for common data dependencies, a stall when an instruction needs a value from the immediately preceding load, and a flush when a taken branch or jump changes the PC. I will first test a manageable set of arithmetic, logical, load/store and branch instructions, then expand instruction coverage if time permits.

Figure 1. Proposed five-stage processor block diagram



Hazard control: forwarding, load-use stall and branch flush

RTL Implementation

I will reuse and adapt the relevant Verilog modules from my existing RISC-V v2 project. I will add the pipeline registers and the forwarding, stall and flush logic. I will check the behavior in simulation using small programs and directed test cases, including dependent instructions, loads followed by dependent operations, and taken and untaken branches.

RTL-to-GDSII Flow

Once the RTL passes simulation, I will synthesize the processor using the standard-cell library and constraints available in the lab. I will inspect the area and timing reports, then run floorplanning, placement, clock-tree synthesis and routing. Finally, I will review the post-route timing, run the physical checks supported by the lab flow, and export the GDSII file.

Expected Results

I expect to submit a verified pipelined processor RTL, simulation results for the key instructions and hazards, synthesis area and timing reports, a routed layout and a GDSII file. I will report the measured results from the tools, including any remaining timing or DRC issues, instead of assuming values in advance.

4. Timeline

Week 1: Review my existing RISC-V v2 code, decide the instruction set for this project, and add the five pipeline stages and registers.

Week 2: Add forwarding, load-use stall and branch flush logic. Run basic instruction simulations.

Week 3: Finish directed tests and fix RTL issues. Run synthesis and review area and timing reports.

Week 4: Run floorplanning, placement, clock-tree synthesis and routing; correct problems found by the tools.

Week 5: Check final timing and physical verification results, export GDSII, and prepare the report and screenshots.